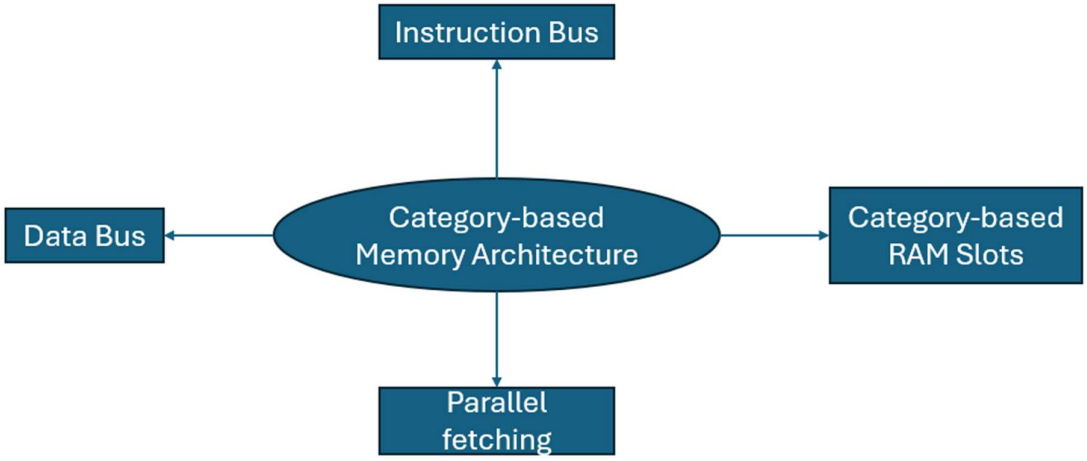
Category-Based Memory Architecture

Contents

Introduction

This Category-Based Memory Architecture is designed to increase the efficiency of fetching within the CPU's FDE cycle. This category-based architecture was developed for two main reasons. The first reason was to address the von Neumann bottleneck with fetching data and instructions at the same time. The second reason is to address the scalability and flexibility limitations of the Havard architecture. This category-based architecture does not change the fundamental principle of using buses or fetching data/instructions from memory. Instead, this architecture focuses on how memory is organised and fetched for both data and instructions.

This category-based memory architecture cannot promise perfect performance, reliability or efficiency. However, it can promise to try improve performance and efficiency as much as possible within the limitations of classical computing. This category-based memory architecture is targeted mainly towards general-purpose systems but can be used and adapted to specific use cases like embedded systems. The technical specifications and details are made for engineers attempting to implement this architecture. Make sure to adapt the fundamental principles of this architecture to the specific use case.



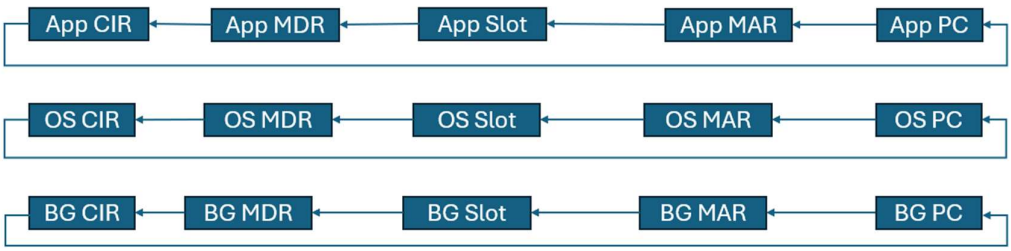
The spider diagram shows the 4 main concepts involved in this category-based memory architecture. For those who want to know more about category-based memory RAM slots, you can look at my motherboard architecture documentation.

Category-Based Memory Architecture Design

- **This category-based memory architecture requires changed to buses, the CPU and the motherboard.**
- **I will first be discussing the changes to the bus architecture.**
- **Each category within a system will have two buses which are the instruction and data buses.**
- **Buses will use sizes such as 64-bits, 128-bits or 256-bits but I would recommend 64-bit for most use cases especially general-purpose systems.**
- **Having 2 buses for each category means that the CPU can fetch both instructions and data from each category without having to wait for other categories.**
- **For example, for the 3 categories of OS, App and BG there would be 6 buses in total to allow for parallel fetching.**
- **To implement these buses, 2 physical traces will need to be added for each RAM slot for the 3 categories system.**
- **This bus architecture is efficient as it allows the CPU to fetch instructions and data at the same time while also allowing for parallel fetching in all categories.**
- **I will now be discussing an overview of the CPU changes needed.**
- **First, the CPU needs to be able to fetch both data and instructions at the same time using the instruction bus and data bus.**
- **This is already achievable due to the Havard's architecture separate buses for data and instructions.**
- **Second, the CPU needs to be able to perform parallel fetching across all categories correctly.**
- **The CPU can be configured to check the available categories and send signals to all buses to fetch data at the same time.**
- **These changes are important because they increase efficiency and allow the CPU to work in parallel instead sequentially.**
- **You can find more details specifically on CPU architecture with category-based memory in its specific documentation.**
- **I will now be discussing the overview of the motherboard changes needed.**
- **First, the motherboard needs to support different RAM slots based on categories in the system.**
- **The order of RAM slots should be decided based on physical distance to the CPU to make sure that signals can reach certain categories faster.**
- **For example, OS and App slots would be closer to the CPU compared to the BG slots for faster fetching across physical distance.**
- **Second, the motherboard must include the physical traces to connect the RAM slots, cache and the CPU together.**
- **The number of physical traces depend on the number of RAM slots while the traces needed for cache and the CPU should be more predictable.**
- **You can read more about the motherboard architecture in its documentation.**

Category-based Memory Architecture Workflow

- **This workflow will cover the whole CPU cycle and how it will work daily using this architecture.**
- **First, the 3 program counters show the next addresses in memory OS: 0x00007C1E, App: 0x00003A00 and BG: 0x00001F2A.**
- **Second, these addressed are stored in the 3 memory address registers until the address buses are available.**
- **Third, these 3 addresses are used by the 3 address buses to find the instruction within the category.**
- **The OS address bus finds the instruction at 0x00007C1E, App address bus finds the instruction 0x00003A00 and BG address bus finds the instruction 0x00001F2A.**
- **Fourth, the data bus and the instruction bus for each category transport instructions and data to the MDR.**
- **Fifth, these data and instructions are stored in the 3-way MDR per category temporarily.**
- **Sixth, the data and instructions are moved to the 3-way CIR for decoding and execution.**
- **Seventh, each core within the CPU is allocated to a category and handles the execution and decoding of each category's instruction.**
- **Eighth, each program counter must be incremented by 1 and move onto to pointing to the next instruction.**
- **The FDE cycle is repeated once again, and the processes above happen again for the next set of instructions.**



The spider diagram above shows the hierarchy of the FDE cycle for each individual category. We can see that it starts at the program counter and ends at the current instruction register. After it reaches the current instruction register and executes the FDE cycles begins back at the program counter once again.

Category-based Memory Architecture Trade-offs

- **This category-based memory architecture has trade-offs just like any other system.**
- **Engineers and developers may face further implementation challenges as this is a completely new architecture.**
- **I have identified 5 main trade-offs with this architecture.**
- **The first trade-off is heat.**
- **This trade-off exists as this architecture is designed to be extremely efficient and reliable.**
- **This is because this architecture is expected to perform 3 FDE cycles simultaneously and use parallel fetching, decoding and executing.**
- **This trade-off can be managed by using heat sinks, fans and vents to ensure that the device or system does not overheat.**
- **The second trade-off is complexity.**
- **This trade-off exists as this architecture uses multiple extra components such as registers and RAM slots as well the logic needed to manage these components.**
- **This means that engineers and developers may find it difficult or confusing to implement this architecture correctly.**
- **This trade-off can be managed by understanding the changes for each component, ensuring roles and responsibilities are clear and defining any unclear details.**
- **The third trade-off is time-consumption.**
- **This trade-off exists as this architecture requires a massive number of changes to physical and logical components.**
- **This means that engineers and developers will need have a coordinated and consistent effort across software, firmware and hardware developments.**
- **This trade-off can be managed by using updating code where possible, using pre-existing components and avoiding unnecessary complexity or features.**
- **The fourth trade-off is adoption**
- **This trade-off exists as this architecture won't be adopted easily and quickly by everyone due to the massive changes .**
- **This means that full on implementation globally could be only fully adopted in years.**
- **This trade-off can be managed by phased implementation, implementing this architecture in updates where possible**
- **The fifth trade-off is cost**
- **This trade-off exists as it will be expensive to get the necessary hardware components and hire the developers and engineers.**
- **This means that it will require a massive budget for a project of this size.**
- **This trade-off can be managed through budget management and being efficient with the changes made.**